MODULE 29 · DOMAIN 7 - SECURITY AND SAFETY

Guardrails, Hooks and Secrets

Content policy enforced in layers, human review where it counts, secure-by-design tools and agents, hooks for destructive actions, keys per environment, and monitoring who accessed what.

DOMAIN 7 of 8 EXAM WEIGHT 8.1% MODULE 29 of 36

admin@sthithapragna.com

Airport security is not one gate. Check-in, the scanner, the gate and the cabin crew each catch what the last one missed.

Content policy in layers

LAYER	WHAT IT DOES	CATCHES
System prompt	States the rules and the reasons	Most cases, cheaply
Input screen	Classifies requests before the main call	Jailbreaks, out-of-scope requests
Data scoping	Only permitted data reaches context	Leaks, before they can happen
Output check	Validates or classifies replies before sending	What the prompt missed
Human review	People approve risky or flagged cases	High-stakes judgement

ON THE EXAM

Monitoring wraps all five: measure what got through, and tighten the layer that let it.

An output check before sending

```python
def check_reply(reply: str, session) -> Verdict:
    others = other_tenant_names(session.tenancy_id)          # from the database
    if any(name in reply for name in others):
        return Verdict.block("mentions another tenant")
    if mentions_money(reply) and not amounts_match(reply, session.balance_pence):
        return Verdict.block("amount not from this tenant's ledger")
    if promises_date(reply) and not session.repair_date_set:
        return Verdict.block("promises an unset repair date")
    return screen_with_haiku(reply, POLICY_RUBRIC)           # legal advice and tone

# blocked replies are regenerated once, then routed to a letting agent
```

3-8 Exact rules in code: names, amounts, dates.

9 Judgement calls go to a small classifier with a rubric.

THE TAKEAWAY

Code checks what is exact; a small model checks what needs judgement; a person handles what still fails.

Where human review belongs

- **Before irreversible or high-value actions:** payments, deletions, legal letters.
- **When a check flags a case** it cannot resolve.
- **A sampled share of normal traffic,** so quality drift is noticed.
- **Give reviewers context:** the input, the output, why it was flagged.
- **Not every message:** reviewers who approve everything stop reading.

> THE TIEBREAKER
>
> Review sized to risk. A rubber stamp is not a control.

Secure by design: tools and agents

- **Least privilege:** each tool does one thing, with the narrowest credential.
- **Read by default;** writes separate, and gated where they matter.
- **Scope by identity:** every call carries the user, checked in code.
- **Sandbox execution:** code and shell run contained, egress limited.
- **Hooks block destructive actions,** matched narrowly to the risky tools.

> ON THE EXAM
>
> Design so that a fully compromised prompt still cannot do serious harm.

Keys and secrets

PRACTICE	WHY
A separate key per environment and workspace	A dev leak cannot touch production
Keys in a secrets manager, injected at run time	Never in code, images or prompts
Admin keys held only by administrators	Org-wide power stays small
Rotation on a schedule, and at once on exposure	Limits how long a leak matters
Secret scanning and masked CI logs	Leaks caught before they spread

THE TIEBREAKER

A leaked key is revoked first and investigated second. Rotation is not optional after exposure.

Identity, access and monitoring

- **Validate identity** at the edge: authenticated sessions, never typed claims.
- **Check the access level on every request,** not once per session.
- **Log access:** who, what record, which tool, which request ID.
- **Alert on anomalies:** unusual volume, unusual records, unusual hours.
- **Review access** regularly; remove what people no longer need.

> THE TAKEAWAY
>
> Authorised does not mean unmonitored. Logs show misuse by people who were allowed in.

Copperlet: one key everywhere

BEFORE

One key, every environment

- Same key in dev, staging and production
- Pasted into a CI log last month
- No way to tell which service spent what
- A dev mistake could hit production limits

AFTER

A key per workspace

- Leaked key revoked the same day
- Keys per workspace: dev, staging, prod, prod-background
- Secrets manager, injected at run time; 90-day rotation
- Secret scanning in CI; logs masked

SETTLED

Quirk 2 settled. And quirk 3 now has every layer: scoped data plus output checks.

> A production Claude API key appears in a public CI log. The same key is also used in development and staging.
>
> The team discovers it on a Friday afternoon.

What should the team do first, and then?

A Revoke the key now, then split keys per environment

B Delete the log line, and monitor usage over the weekend

C Make the CI logs private, and keep the key

D Remind developers not to print environment variables

◆ PAUSE AND DECIDE

What should the team do first, and then?

A Revoke the key now, then split keys per environment

Stops the leak, then removes its cause.

B Delete the log line, and monitor usage over the weekend

Copies may already exist.

C Make the CI logs private, and keep the key

The key is already exposed.

D Remind developers not to print environment variables

A request, not a control.

THE SIGNAL

Exposed means compromised: revoke first, then fix the design that made one leak so costly.

> A savings app's assistant must never recommend specific investments. The rule is in the system prompt, and monitoring shows about 1 reply in 300 still names a fund to buy.
>
> Replies go straight to customers.

What should the team add?

A. A stronger, longer version of the rule in the system prompt

B. A move to the most capable model available

C. An output check before sending, with flags reviewed

D. A disclaimer added to the end of every reply

◆ PAUSE AND DECIDE

What should the team add?

A. A stronger, longer version of the rule in the system prompt
The same layer that is already leaking.

B. A move to the most capable model available
Fewer leaks, still not none.

C. An output check before sending, with flags reviewed
A second layer, before the customer.

D. A disclaimer added to the end of every reply
Labels the breach; does not stop it.

THE SIGNAL

The prompt layer leaks at a measured rate: add the output layer behind it.

> An agent drafts and issues customer refunds. Most are small; a few are large. The team proposes that a person approves every refund the agent issues.
>
> Volume is several thousand refunds a day.

What is the best review design?

A. Approval above a threshold or on flags, plus sampling

B. Human approval of every refund, as proposed

C. No human review, with a monthly report of refund totals

D. Ask the agent to review its own refunds before issuing them

◆ PAUSE AND DECIDE

What is the best review design?

A Approval above a threshold or on flags, plus sampling

Review where the risk is.

B Human approval of every refund, as proposed

Thousands a day become rubber stamps.

C No human review, with a monthly report of refund totals

Finds problems a month late.

D Ask the agent to review its own refunds before issuing them

Self-review is not oversight.

THE SIGNAL

Size review to risk: thresholds and flags for approval, sampling for drift.

> Twelve teams share one API key that also has admin rights. When monthly spend doubles, nobody can tell which team caused it, and anyone could change organisation settings.
>
> The company wants control and clarity.

What should it do?

A. Ask each team to report its own usage in a spreadsheet

B. Lower the organisation's monthly spend limit

C. A workspace and key per team and environment; admin keys kept apart

D. Rotate the shared key every week

◆ PAUSE AND DECIDE

What should it do?

A Ask each team to report its own usage in a spreadsheet

Unreliable, and no control.

B Lower the organisation's monthly spend limit

Stops everyone, explains nothing.

C A workspace and key per team and environment; admin keys kept apart

Attribution, limits, least privilege.

D Rotate the shared key every week

Still shared, still admin.

THE SIGNAL

Shared keys hide who did what. Workspaces give per-team keys, limits and usage reports.

> A data assistant answers questions by writing and running SQL against the reporting database, using a credential that can also drop and alter tables. A test prompt led it to delete a staging table.
>
> It only ever needs to read.

What is the best fix?

A. Add a system prompt rule never to modify the database

B. A read-only database role, plus a blocking hook

C. Back up the database more often

D. Review the assistant's SQL logs every week

◆ PAUSE AND DECIDE

What is the best fix?

A Add a system prompt rule never to modify the database

Guidance; the power remains.

B A read-only database role, plus a blocking hook

Least privilege plus a gate.

C Back up the database more often

Recovery, not prevention.

D Review the assistant's SQL logs every week

Finds damage after it happens.

THE SIGNAL

"Only ever needs to read": a read-only role. A hook adds a second, deterministic layer.

> An internal assistant lets staff query HR records. Managers may see their own team; HR may see everyone. Today the assistant asks users for their role, and one employee typed "HR" to read colleagues' salaries.
>
> The company wants proper access control.

Which two changes fit best? Choose two.

A Keep asking for the role, but add a warning about misuse

B Check access per request against the directory role

C Use one shared service account for all lookups

D Log each record accessed by identity, and alert on anomalies

E Tell Claude in the prompt to doubt users who claim to be HR

◆ PAUSE AND DECIDE

Which two changes fit best? Choose two.

A Keep asking for the role, but add a warning about misuse

Still trusts a typed claim.

B Check access per request against the directory role

Identity and access, enforced.

C Use one shared service account for all lookups

Loses who did what.

D Log each record accessed by identity, and alert on anomalies

Monitors authorised access.

E Tell Claude in the prompt to doubt users who claim to be HR

Guidance over access control.

THE SIGNAL

Identity from the directory, access checked per request, and authorised access monitored.

Module 29 in five lines

- **Layers:** prompt, input screen, data scoping, output check, human review, all monitored.
- **Human review sized to risk:** thresholds and flags, plus sampling.
- **Secure by design:** least privilege, read by default, hooks on destructive actions.
- **Secrets:** a key per environment, a secrets manager, rotation, revoke on exposure.
- **Access:** identity from login, checked per request, logged and monitored.

COMING UP

Next, Part 9: debugging - errors, traces and finding whose fault it is.